

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



AN NINH MÁY TÍNH
23CNTTHUC

Project 01 MINI VAULT

Lương Văn Dũng_23127353

Nguyễn Văn Khánh_23127388

Trần Hữu Nghĩa_23127437

MỤC LỤC

1. Thông tin nhóm và Phân công công việc	3
2. Sơ đồ kiến trúc hệ thống	5
2.1. Cấu trúc thư mục dự án	5
2.2. Sơ đồ kiến trúc	6
3. Giải thích Kỹ thuật	8
3.1. Khởi tạo và Mở khóa Vault (Feature 0.1)	8
3.2. Định danh và Xác thực Người dùng (Feature 0.2).....	10
3.3. Mã hóa tĩnh và Toàn vẹn dữ liệu - KV Engine (Feature 1.1).....	13
3.4. Phân quyền truy cập dựa trên Sở hữu (Feature 1.2)	16
3.5. Quản trị Khóa định danh (Feature 2.1)	18
3.6. Dịch vụ Mã hóa và Giải mã - Transit Engine (Feature 2.2).....	21
3.7. Phân quyền Truy cập Khóa định danh (Feature 2.3)	24
3.8. Dịch vụ Ký và Xác thực Chữ ký số (Feature 2.4).....	25
4. Tính năng Extra Credit.....	28
4.1. Secret Versioning.....	28
4.2. Key Rotation	30
4.3. Tamper-evident Audit Log	32
5. Kết luận và Giới hạn triển khai	35
5.1. Tổng kết.....	35
5.2. Giới hạn triển khai.....	35
Tài liệu tham khảo.....	36

1. Thông tin nhóm và Phân công công việc

Danh sách thành viên:

- Lương Văn Dũng - 23127353
- Nguyễn Văn Khánh - 23127388
- Trần Hữu Nghĩa - 23127437

Bảng phân công công việc:

Ứng với mã số sinh viên là tên các thành viên như chú thích phía trên.

MSSV	Nhiệm vụ	Mức độ hoàn thành
23127353	- Xây dựng kiến trúc Backend (FastAPI, SQLAlchemy) - Cài đặt Feature 0 (Vault Initialization & User Auth) - Cài đặt Feature 2.1 – 2.2(Transit Engine - Encrypt/Decrypt) - Cài đặt Tính năng nâng cao: Key Rotation (Transit Engine) - Xây dựng Core Crypto: AES-GCM, Argon2id KDF	100%
23127388	- Thiết kế Database Models - Cài đặt Feature 1 (Secure Storage KV Engine) - Quay video demo - Chụp ảnh minh họa trong báo cáo - Cài đặt Tính năng nâng cao: Secret Versioning (KV Engine)	100%
23127437	- Cài đặt Feature 2.3 – 2.4 (Sign/Verify) - Test API - Viết Báo cáo kỹ thuật - Cài đặt Tính năng nâng cao: Tamper-evident Audit Log	100%

Link Video demo: https://www.youtube.com/watch?v=Sud6r7sF_Gk

Video được ghi hình trực tiếp dựa trên kịch bản chạy tuần tự thông qua API (sử dụng cấu hình từ file docs/api-demo.http), bao gồm các bước sau:

1. **Khởi tạo và Mở khóa (Feature 0.1):** Thực hiện Init và Unlock Vault bằng Master Passphrase.
2. **Định danh (Feature 0.2):** Đăng ký tài khoản cho 2 người dùng (Alice và Bob), thực hiện Login để nhận Session Token.
3. **Lưu trữ An toàn và Phân quyền (Feature 1):** Alice ghi và đọc bí mật tại secret/alice@example.com/database. Đổi sang Token của Bob để truy cập và bị chặn lập tức (lỗi 403 Permission Denied).
4. **Mã hóa Transit và Cách ly quyền (Feature 2.1 & 2.2):** Alice tạo khóa payment-key, thực hiện Encrypt và Decrypt văn bản. Dùng Token của Bob để dùng thử khóa này và bị hệ thống từ chối.
5. **Ký và Xác thực Chữ ký số (Feature 2.4):** Alice tạo khóa document-signing-key, thực hiện Ký (Sign) và Xác thực (Verify) thông điệp thành công.
6. **Chống giả mạo:** Chủ đích sửa đổi 1 byte trong thông điệp và đưa vào hàm Verify. Hệ thống lập tức trả về signature_valid: false (Bắt lỗi thành công).
7. **Khóa Vault:** Gọi lệnh Lock Vault. Các thao tác KV/Transit sau đó đều bị từ chối với mã lỗi HTTP 423 VAULT_LOCKED.

2. Sơ đồ kiến trúc hệ thống

2.1. Cấu trúc thư mục dự án

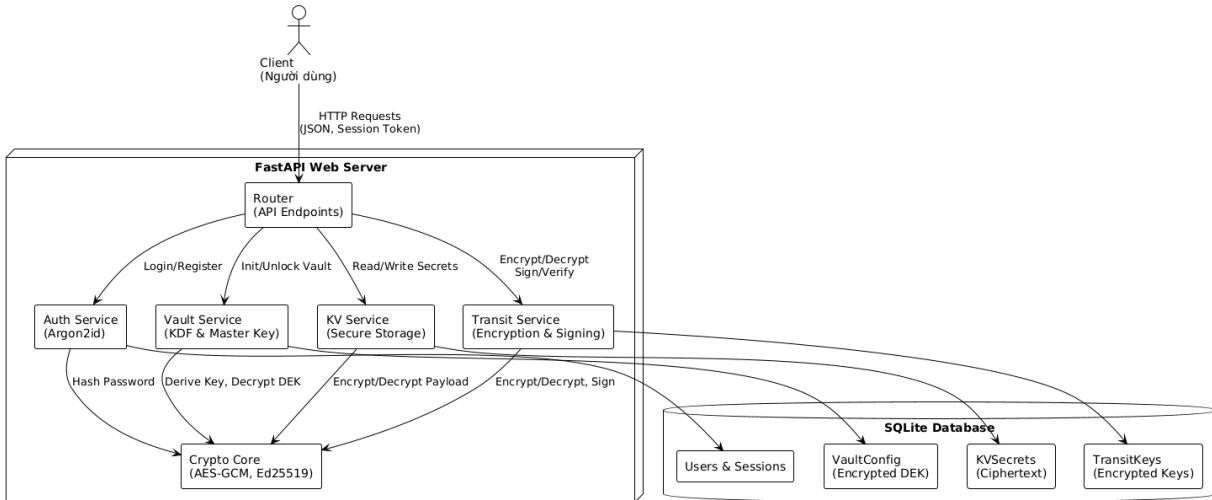
Dự án được tổ chức theo chuẩn kiến trúc của một ứng dụng FastAPI hiện đại (Package-based), tách biệt rõ ràng giữa logic định tuyến, nghiệp vụ và truy xuất dữ liệu nhằm tối ưu hóa tính toàn vẹn và khả năng bảo trì:

```
Mini-Vault/
├── app/
│   ├── auth/
│   │   └── service.py
│   ├── core/
│   │   ├── crypto_service.py
│   │   ├── key_derivation.py
│   │   └── vault_state.py
│   ├── kv/
│   │   └── service.py
│   ├── transit/
│   │   └── service.py
│   ├── utils/
│   │   └── validation.py
│   ├── router.py
│   ├── dependencies.py
│   ├── models.py
│   └── database.py
├── data/
│   └── mini_vault.db
├── docs/report/
├── tests/
└── requirements.txt
```

Chứa toàn bộ mã nguồn cốt lõi (Source Code)
Business Service: Quản lý đăng ký, đăng nhập
Xử lý băm Argon2id và kiểm duyệt Password
Crypto Core & Vault Service
Triển khai thuật toán AEAD (AES-256-GCM)
Thuật toán KDF dẫn xuất khóa từ Passphrase
Quản lý vòng đời DEK trên RAM (Root of Trust)
Business Service: Feature 1 (Secure Storage)
CRUD bí mật với cơ chế Secret Versioning
Business Service: Feature 2 (Transit Engine)
Quản trị khóa, mã hóa và ký số (Ed25519)
Các hàm tiện ích hỗ trợ toàn cục
Ngăn chặn Path Enumeration và kiểm duyệt Payload
API Gateway định tuyến và ghi nhật ký Audit Log
Middleware chặn truy cập trái phép bằng Session Token
Định nghĩa Schema cho cơ sở dữ liệu (SQLAlchemy)
Quản lý kết nối Storage Layer (SQLite)
Storage Layer: Vị trí lưu trữ cơ sở dữ liệu vật lý
File CSDL (luôn ở trạng thái Encrypted-at-Rest)
Thư mục chứa tài liệu báo cáo và minh chứng
Phân hệ kiểm thử tự động (Unit Test với Pytest)
Danh sách các thư viện phụ thuộc
(cryptography, fastapi...)

Ghi chú về Hướng dẫn cài đặt và khởi chạy hệ thống: Chi tiết các bước thiết lập môi trường ảo, lệnh cài đặt các thư viện phụ thuộc, lệnh khởi chạy máy chủ FastAPI cũng như cách thức truy cập giao diện Swagger UI để kiểm thử API đã được nhóm trình bày đầy đủ tại file **README.md** nằm ở thư mục gốc của dự án.

2.2. Sơ đồ kiến trúc



Kiến trúc của hệ thống Mini-Vault được thiết kế theo mô hình **Phân tầng** kết hợp với nguyên tắc **Đặc quyền tối thiểu**, giúp cô lập chặt chẽ giữa logic định tuyến, nghiệp vụ mã hóa và lưu trữ vật lý. Các Layer gồm:

Client-Server Interface:

- **Client:** Giao tiếp với hệ thống thông qua các HTTP Requests. Mọi payload dữ liệu đều được đóng gói dưới dạng chuẩn JSON và xác thực bằng Session Token đính kèm trong Header.
- **Router:** Đóng vai trò là API Gateway duy nhất của FastAPI Web Server. Lớp này chịu trách nhiệm tiếp nhận yêu cầu, kiểm duyệt tính hợp lệ của Token thông qua Dependency Injection trước khi điều phối luồng dữ liệu xuống các dịch vụ bên dưới.

Business Services: Hệ thống phân rã nghiệp vụ thành 4 khối Service hoạt động độc lập, đảm bảo tính Module hóa cao:

- **Auth Service:** Tiếp nhận luồng Login/Register, chịu trách nhiệm quản lý vòng đời tài khoản và Session.
- **Vault Service:** Xử lý luồng Init/Unlock, đóng vai trò kích hoạt trái tim của hệ thống bằng cách khởi tạo và giải phóng DEK lên RAM.

- **KV Service:** Tiếp nhận luồng Read/Write Secrets, thực thi quy trình Access Control cực kỳ nghiêm ngặt trước khi cho phép truy cập bí mật.
- **Transit Service:** Cung cấp mô hình Cryptography-as-a-Service, chịu trách nhiệm Key Wrapping và xử lý các luồng Encrypt/Decrypt, Sign/Verify mà không bao giờ làm lộ khóa ra ngoài.

Crypto Core: Đây là bộ não mật mã học của toàn bộ hệ thống, tách biệt hoàn toàn khỏi logic nghiệp vụ. Khi các Service cần thao tác dữ liệu an toàn, chúng bắt buộc phải gọi xuống Crypto Core. Tầng này tập trung triển khai các thuật toán tiêu chuẩn cao:

- Băm mật khẩu bằng **Argon2id** (được gọi bởi Auth Service).
- Dẫn xuất KDF và giải mã DEK (được gọi bởi Vault Service).
- Mã hóa AEAD bằng **AES-256-GCM** (được gọi bởi KV và Transit Service).
- Ký số EdDSA bằng **Ed25519** (được gọi bởi Transit Service).

Storage Layer: Toàn bộ dữ liệu được lưu trữ tập trung tại cơ sở dữ liệu **SQLite**. Kiến trúc đảm bảo mô hình **Encrypted-at-Rest** ở cấp độ cao nhất:

- Bảng Users & Sessions chỉ lưu trữ chuỗi băm của mật khẩu và token.
- Bảng VaultConfig chứa DEK đã bị Key Wrapping bởi Master Passphrase.
- Bảng KVSecrets và TransitKeys tuyệt đối không lưu trữ bất kỳ chuỗi plaintext nào, mọi cấu trúc đều là Ciphertext tự mô tả, đảm bảo vô hiệu hóa mọi rủi ro rò rỉ kể cả khi hacker đánh cắp được cơ sở dữ liệu vật lý.

3. Giải thích Kỹ thuật

3.1. Khởi tạo và Mở khóa Vault (Feature 0.1)

Cấu trúc mã nguồn liên quan:

- **Routing API:** app/router.py (Nhóm endpoint /api/v1/vault/*)
- **Business Logic:** app/core/vault_service.py (Thực thi nghiệp vụ cốt lõi qua các hàm VaultService.init và VaultService.unlock)
- **State Management:** app/core/vault_state.py (Quản lý trạng thái DEK trên RAM qua lớp VaultState và hàm require_dek())
- **Cryptography & Validation:** app/core/key_derivation.py (Dẫn xuất khóa Master Passphrase bằng hàm derive_key()) và app/utlis/validation.py (Kiểm định độ phức tạp bằng hàm validate_passphrase())

Mục tiêu cốt lõi: Tính năng này thiết lập Root of Trust cho toàn bộ kiến trúc Mini-Vault. Nhiệm vụ tối thượng của nó là đảm bảo Khóa mã hóa DEK được sinh ra an toàn, được bảo vệ nghiêm ngặt bằng mật mã học khi lưu trữ tĩnh, và chỉ được giải phóng vào bộ nhớ RAM khi có sự can thiệp hợp lệ từ người quản trị. Nội dung tính năng này gồm các kỹ thuật chính như sau:

a. Khởi tạo Master Passphrase và bảo vệ Data Encryption Key: Data Encryption Key (DEK) đóng vai trò là khóa mã hóa chủ đạo nội bộ, thao tác trực tiếp lên mọi luồng mã hóa dữ liệu hạ tầng. Khởi chức năng VaultService.init được thiết kế để tuyệt đối không lưu trữ DEK dưới định dạng plaintext mà triển khai luồng mã hóa phân tầng:

- Tại bước thiết lập, khi quản trị viên cung cấp Master Passphrase thông qua API, chuỗi đầu vào lập tức được chuyển qua hàm validate_passphrase(). Hàm này áp đặt cơ chế kiểm định độ phức tạp nghiêm ngặt (tối thiểu 12 ký tự, bắt buộc chứa chữ hoa, chữ thường, chữ số và ký tự đặc biệt) đồng thời yêu cầu xác nhận kép nhằm loại trừ rủi ro sai sót thao tác.
- Sau khi xác thực hợp lệ, hệ thống phát sinh chuỗi Salt ngẫu nhiên 16-byte và đưa Master Passphrase qua hàm KDF derive_key() (cốt lõi sử dụng thuật toán Argon2id). Cấu hình Argon2id được tinh chỉnh nhằm tăng cường chi phí bộ nhớ và thời gian tính toán, vô hiệu hóa các kỹ thuật tấn công Brute-force. Kết quả trả về là một khóa đối xứng 256-bit.

- Khóa dẫn xuất này lập tức được đưa vào thuật toán AEAD AES-256-GCM để mã hóa khối DEK 32-byte vừa được sinh ngẫu nhiên.
- Bản mã của DEK cùng Salt và cấu hình KDF Parameters được lưu trữ định danh vào bảng VaultConfig. Toàn vẹn hệ mật nhờ đó được bảo đảm độc lập với rủi ro lộ lọt cơ sở dữ liệu vật lý.

b. Cơ chế Mở khóa và Bắt lỗi bằng Authentication Tag

- Do tính chất giải phóng vùng nhớ của RAM, lớp quản lý trạng thái VaultState luôn mặc định gán DEK bằng null khi khởi động. Mọi luồng truy xuất dữ liệu từ các dịch vụ khác tại thời điểm này đều bị chặn đứng bởi hàm require_dek() với ngoại lệ VAULT_LOCKED.
- Khi quản trị viên yêu cầu mở khóa hệ thống thông qua hàm VaultService.unlock và cung cấp Master Passphrase, hệ thống dẫn xuất lại khóa và tiến hành giải mã. Đặc tả của AES-256-GCM tích hợp sẵn thành phần Authentication Tag 16-byte đính kèm bản mã nhằm xác thực tính toàn vẹn.
- Nếu Master Passphrase cung cấp không hợp lệ, khóa dẫn xuất sẽ bị sai lệch. Quá trình đối chiếu Authentication Tag của thuật toán AES-GCM lập tức thất bại và ném ra ngoại lệ InvalidTag.
- Lớp nghiệp vụ chủ động kiểm soát ngoại lệ InvalidTag và quy đổi thành Generic Error mang mã INVALID_MASTER_PASSPHRASE. Kỹ thuật này ngăn chặn triệt để sự hình thành DEK rác có nguy cơ phá vỡ cấu trúc mã hóa, đồng thời triệt tiêu rủi ro rò rỉ thông tin (Information Leakage) thông qua các thông báo lỗi tăng thấp.

```
khanhvn2281 - zsh - 0.1 - init & unlock

→ Mini-Vault git:(main) x curl -s http://localhost:8321/api/v1/vault/status
{"success":true,"data":{"initialized":false,"status":"locked"},"error":null}%

→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/vault/init \
-H "Content-Type: application/json" \
-d '{"master_passphrase":"Strong-Master-Passphrase-2026!","confirm_master_passphrase":"Strong-Master-Passphrase-2026!"}'
{"success":true,"data":{"initialized":true,"status":"locked"},"error":null}%

→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/vault/unlock \
-H "Content-Type: application/json" \
-d '{"master_passphrase":"sai-passphrase-bat-ky"}'
{"success":false,"data":null,"error":{"code":"INVALID_MASTER_PASSPHRASE","message":"Invalid master passphrase"}}%

→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/vault/unlock \
-H "Content-Type: application/json" \
-d '{"master_passphrase":"Strong-Master-Passphrase-2026!"}'
{"success":true,"data":{"status":"unlocked"},"error":null}%

→ Mini-Vault git:(main) x
```

Ảnh 3.1.1: Khởi tạo Vault thành công và trả về thông tin cấu hình

```
khanhvn2281 - zsh - 0.1 - lock lại vault

→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/vault/lock
{"success":true,"data":{"status":"locked"},"error":null}%

# gọi lại KV/Transit sau khi lock -> phải bị chặn (HTTP 423)
→ Mini-Vault git:(main) x curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8321/api/v1/kv/secret/
alice@example.com/database \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4ft0QA4bhap30THU"
423%

→ Mini-Vault git:(main) x
```

Ảnh 3.1.2: Hệ thống từ chối truy cập với lỗi VAULT_LOCKED khi hệ thống chưa được mở khóa

3.2. Định danh và Xác thực Người dùng (Feature 0.2)

Cấu trúc mã nguồn liên quan:

- **Routing API:** app/router.py (Nhóm endpoint /api/v1/auth/*)
- **Business Logic:** app/auth/service.py (Thực thi nghiệp vụ định danh qua các hàm AuthService.register và AuthService.login)
- **Token Security:** app/auth/service.py (Khai triển băm Session Token bằng hàm token_digest())

Mục tiêu cốt lõi: Tính năng này thiết lập Identity Perimeter độc lập cho từng người dùng. Trước khi bất kỳ thao tác tương tác dữ liệu nào được cho phép, hệ thống phải xác thực danh tính nhằm ngăn chặn truy cập trái phép, đồng thời chủ động chống lại các kỹ thuật tấn công dò quét mật khẩu tự động như Credential Stuffing.

Kiến trúc định danh của hệ thống được vận hành dựa trên các tiêu chuẩn bảo mật khắt khe như sau:

a. Đăng ký tài khoản và Mã hóa Password: Khởi chức năng AuthService.register tiếp nhận thông tin khởi tạo tài khoản thông qua API. Tương tự Master Passphrase, mật khẩu người dùng bị áp đặt bộ quy tắc kiểm định độ phức tạp thông qua hàm validate_passphrase(). Thay vì lưu trữ plaintext, hệ thống triển khai hàm băm một chiều thông qua lớp đối tượng PasswordHasher để băm mật khẩu người dùng. Lựa chọn thuật toán Argon2 thay cho cấu trúc SHA thuần túy nhằm tối ưu hóa khả năng chống chịu trước các kỹ thuật tấn công Brute-force và Rainbow Table nhờ đặc tính kháng GPU và kháng phần cứng chuyên dụng ASIC của Argon2id.

b. Cơ chế Đăng nhập và Ngăn chặn Brute-force: Khi người dùng thực thi API đăng nhập, hàm AuthService.login tiến hành truy vấn cơ sở dữ liệu và xác thực mật khẩu thông qua thủ tục đối chiếu mã băm. Nếu phát sinh ngoại lệ VerifyMismatchError từ thư viện Argon2, hệ thống lập tức tăng biến đếm failed_login_attempts trên mô hình dữ liệu User. Nhằm kiểm soát rủi ro dò mật khẩu, hệ thống kích hoạt cơ chế Lockout chủ động. Khi số lần đăng nhập thất bại liên tiếp đạt ngưỡng 5, thuộc tính locked_until được gán mốc thời gian tịnh tiến 5 phút. Mọi luồng đăng nhập tiếp theo trong khung thời gian này đều bị hàm login từ chối bằng ngoại lệ ACCOUNT_LOCKED, bất kể tính hợp lệ của mật khẩu được cung cấp sau đó.

c. Quản trị Phiên làm việc bằng Session Token: Sau khi xác thực thành công, định danh người dùng được duy trì thông qua Session Token sinh ngẫu nhiên từ hàm secrets.token_urlsafely(). Nhằm thiết lập rào cản bảo mật phòng hờ kịch bản lộ lọt cơ sở dữ liệu vật lý, token nguyên bản chỉ được trả về cho phía client thông qua phản hồi API. Phiên bản lưu trữ tại bảng Session của SQLite bị ép buộc đi qua hàm băm mã hóa token_digest(). Kỹ thuật này triệt tiêu hoàn toàn khả năng phục hồi Session Token hợp lệ từ khối dữ liệu tĩnh.

d. Bảo vệ Endpoint bằng Dependency Injection: Hệ thống sử dụng cơ chế Dependency Injection của FastAPI (Depends(current_principal)) để thiết lập một trạm gác tại tầng định tuyến (app/router.py). Ngoại trừ các API có tính chất toàn cục (như tạo Vault, Login,

Register), 100% các endpoint thuộc nhóm KV Engine và Transit Engine đều bị ép buộc khai báo tham số phụ thuộc này. Bất kỳ yêu cầu nào thiếu Token, gửi Token sai định dạng, hoặc Token đã hết hạn đều bị chặn đứng ngay từ vòng ngoài và trả về mã lỗi UNAUTHENTICATED, bảo vệ tuyệt đối các hàm nghiệp vụ mã hóa bên trong.

```
khanhvv2281 - zsh - 0.2 - register & login

→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"alice@example.com","passphrase":"StrongPassword@123","confirm_passphrase":"StrongPassword@123"}'
{"success":true,"data":{"email":"alice@example.com"},"error":null}%

→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"bob@example.com","passphrase":"StrongPassword@456","confirm_passphrase":"StrongPassword@456"}'
{"success":true,"data":{"email":"bob@example.com"},"error":null}%

# dang ky lai email cu -> phai bi tu choi
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"alice@example.com","passphrase":"StrongPassword@123","confirm_passphrase":"StrongPassword@123"}'
{"success":false,"data":null,"error":{"code":"EMAIL_ALREADY_EXISTS","message":"Email already exists"}}%

# mat khau qua yeu
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"weak@example.com","passphrase":"123456","confirm_passphrase":"123456"}'
{"success":false,"data":null,"error":{"code":"WEAK_PASSPHRASE","message":"Passphrase does not meet security requirements"}}%

# nhap sai mat khau 5 lan lien tiep
→ Mini-Vault git:(main) x for i in 1 2 3 4 5; do curl -s -X POST http://localhost:8321/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"alice@example.com","passphrase":"SaiMatKhau@000"}'; echo; done
{"success":false,"data":null,"error":{"code":"INVALID_CREDENTIALS","message":"Invalid credentials"}}
{"success":false,"data":null,"error":{"code":"INVALID_CREDENTIALS","message":"Invalid credentials"}}
{"success":false,"data":null,"error":{"code":"INVALID_CREDENTIALS","message":"Invalid credentials"}}
{"success":false,"data":null,"error":{"code":"INVALID_CREDENTIALS","message":"Invalid credentials"}}
{"success":false,"data":null,"error":{"code":"INVALID_CREDENTIALS","message":"Invalid credentials"}}%

# dung mat khau that nhung dang trong luc bi khoa
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"alice@example.com","passphrase":"StrongPassword@123"}'
{"success":false,"data":null,"error":{"code":"ACCOUNT_LOCKED","message":"Account is temporarily locked"}}%

→ Mini-Vault git:(main) x
```

Ảnh 3.2.1: Cơ chế Lockout tự động khóa tài khoản khi phát hiện tấn công dò mật khẩu

```
khanhvn2281 - zsh - 0.2 - login sau khi het khoa

# doi 5 phut roi dang nhap lai Alice (dung mat khau)
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"alice@example.com","passphrase":"StrongPassword@123"}'
{"success":true,"data":{"access_token":"3ZfkMX_bzP1kgiF3RbZ3VYaK4S4fT0QAx4bhap30THU","token_type":"Beare
r","expires_in":1800},"error":null}%

# dang nhap Bob
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"bob@example.com","passphrase":"StrongPassword@456"}'
{"success":true,"data":{"access_token":"VGpuYg2_tza2_TQKMyYfUMxuuL4dH_hlkR9msbxD03w","token_type":"Beare
r","expires_in":1800},"error":null}%

→ Mini-Vault git:(main) x
```

Ảnh 3.2.2: Trả về Session Token an toàn sau khi đăng nhập hợp lệ

3.3. Mã hóa tĩnh và Toàn vẹn dữ liệu - KV Engine (Feature 1.1)

Cấu trúc mã nguồn liên quan:

- **Routing API:** app/router.py (Nhóm endpoint /api/v1/kv/*)
- **Business Logic:** app/kv/service.py (Thực thi nghiệp vụ qua các hàm KVService.write, KVService.read và KVService.delete)
- **Cryptography:** app/core/crypto_service.py (Khai triển mã hóa AEAD qua các hàm encrypt() và decrypt())

Mục tiêu cốt lõi: Tính năng này đảm bảo mọi dữ liệu nhạy cảm của người dùng phải được mã hóa tĩnh Encrypted-at-Rest trước khi ghi xuống ổ cứng nhằm chống lại việc rò rỉ dữ liệu khi cơ sở dữ liệu vật lý bị xâm nhập.

Để đáp ứng các tiêu chí trên, hệ thống triển khai tổ hợp các cơ chế phòng vệ dữ liệu tĩnh như sau:

a. Quy trình Lưu trữ bí mật: Khi tiếp nhận yêu cầu lưu trữ thông qua hàm KVService.write, hệ thống lập tức trích xuất DEK từ lớp VaultState. Khối dữ liệu JSON nguyên bản được đưa qua thuật toán mã hóa có xác thực AES-256-GCM. Thay vì sử dụng chung một Nonce tĩnh, hàm encrypt() chủ động phát sinh một chuỗi Nonce 12-byte ngẫu nhiên cho mỗi thao tác ghi nhằm triệt tiêu rủi ro suy diễn khóa Key Recovery Attack. Chuỗi Nonce này được chiết xuất từ hàm os.urandom(12). Authentication Tag có

độ dài 16-bytes và nối trực tiếp vào phần đuôi của bản mã. Khối cấu trúc cuối cùng bao gồm Nonce và [Bản mã + 16-bytes Tag] được lưu trữ định dạng Base64 vào bảng KVSecret.

b. Quy trình Trích xuất và Xác thực: Khi trích xuất dữ liệu qua hàm KVService.read, quy trình giải mã AES-256-GCM sẽ tự động đối chiếu Authentication Tag. Mọi hành vi can thiệp sửa đổi trực tiếp lên file cơ sở dữ liệu SQLite đều làm hỏng cấu trúc Tag, dẫn đến ngoại lệ InvalidTag và hệ thống lập tức từ chối trả về dữ liệu rác, ném ra ngoại lệ mang mã DECRYPTION_FAILED.

c. Cập nhật dữ liệu và Quản lý phiên bản (Secret Versioning): Vượt ra khỏi yêu cầu cơ bản là ghi đè dữ liệu cũ, hệ thống của nhóm đã triển khai thành công tính năng nâng cao Secret Versioning. Khi người dùng thực thi hàm KVService.write lên một đường dẫn đã tồn tại, dữ liệu không bị ghi đè (update) mà tự động được đẩy thành một phiên bản mới, insert vào cơ sở dữ liệu, giữ nguyên vẹn lịch sử thay đổi (Chi tiết kiến trúc này được trình bày tại Mục 4.1). Tương tự, thông qua hàm KVService.delete, hệ thống mới tiến hành xóa cứng toàn bộ lịch sử của bản ghi đó để hủy dữ liệu vĩnh viễn.

d. Xử lý Ngoại lệ và Acceptance Criteria: Mọi thao tác nghiệp vụ đều được kiểm soát bằng hệ thống bắt lỗi chặt chẽ, đáp ứng các tiêu chuẩn:

- **Trạng thái hệ thống:** Bất kỳ thao tác Ghi/Đọc/Xóa nào khi Vault đang khóa đều bị hàm require_dek() chặn đứng ngay lập tức với mã lỗi VAULT_LOCKED.
- **Xử lý sai lệch dữ liệu:**
 - Nếu người dùng cố tình đọc một đường dẫn không tồn tại, cơ sở dữ liệu trả về rỗng, hệ thống ném ngoại lệ NOT_FOUND, tuyệt đối không có dữ liệu rác nào được trả về.
 - Nếu Authentication Tag không khớp trong lúc đọc (chứng tỏ dữ liệu trên đĩa đã bị thay đổi thủ công hoặc can thiệp bằng Hex Editor), hàm decrypt sẽ từ chối giải mã, trả về mã lỗi DECRYPTION_FAILED và không giải phóng bất kỳ plaintext nào.
 - Thử nghiệm bảo mật tệp tĩnh: Khi mở trực tiếp tệp cơ sở dữ liệu SQLite bằng trình soạn thảo văn bản, không có bất kỳ plaintext fragment của bất kỳ bí mật nào có thể bị nhìn thấy do dữ liệu mã hóa đã bị bao bọc trong định dạng Base64.

```
khanhvn2281 - zsh - 1.1 - KV lưu mã hóa tại cho

→ Mini-Vault git:(main) x curl -s -X PUT http://localhost:8321/api/v1/kv/secret/alice@example.com/database \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4fT0QA4bh3p30THU" \
-H "Content-Type: application/json" \
-d '{"data":{"username":"admin","password":"database-password","host":"localhost"}}'
{"success":true,"data":{"path":"secret/alice@example.com/database","created_at":"2026-07-24T15:26:06.936399+00:00","updated_at":"2026-07-24T15:26:06.936399+00:00"},"error":null}%

# doc lai, phai ra dung du lieu goc
→ Mini-Vault git:(main) x curl -s http://localhost:8321/api/v1/kv/secret/alice@example.com/database \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4fT0QA4bh3p30THU"
{"success":true,"data":{"path":"secret/alice@example.com/database","value":{"host":"localhost","password":"database-password","username":"admin"},"error":null}%

# mo file DB bang sqlite3, kiem tra khong co plaintext
→ Mini-Vault git:(main) x sqlite3 data/mini_vault.db "SELECT ciphertext_b64 FROM kv_secrets WHERE path='secret/alice@example.com/database';"
kKKDmaiQfZcQao16Xt7s3YLqWRv8LjwaUqsXsU4ucbi/x3cSB8Eonq0GJ0GcYABhkWLM+1mpe/QD6kU+pF4ownM2IcQ/urv+81IXsVN/0h
afpSxnQcY=%

# sua tay 1 ky tu cuoi cua ciphertext_b64 truc tiep trong DB (mo phong tamper)
→ Mini-Vault git:(main) x sqlite3 data/mini_vault.db "UPDATE kv_secrets SET ciphertext_b64 = substr(ciphertext_b64,1,length(ciphertext_b64)-1) || 'A' WHERE path='secret/alice@example.com/database';"

# doc lai sau khi tamper -> phai bi tu choi
→ Mini-Vault git:(main) x curl -s http://localhost:8321/api/v1/kv/secret/alice@example.com/database \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4fT0QA4bh3p30THU"
{"success":false,"data":null,"error":{"code":"DECRYPTION_FAILED","message":"Secret could not be decrypted"}}%

→ Mini-Vault git:(main) x
```

Ảnh 3.3.1: Dữ liệu dưới Database bị mã hóa hoàn toàn thành các Base64 Ciphertext

```
khanhvv2281 - zsh - 2.5 - tamper ciphertext tren dia

# Alice ghi 1 secret moi de test tamper
→ Mini-Vault git:(main) x curl -s -X PUT http://localhost:8321/api/v1/kv/secret/alice@example.com/tamper-test \
-H "Authorization: Bearer gIzyIbYjY-VfZng2FwVbxSw1CKny0GMEMMTwQjaG2Xs" \
-H "Content-Type: application/json" \
-d '{"data":{"note":"tamper demo"}}'
{"success":true,"data":{"path":"secret/alice@example.com/tamper-test","created_at":"2026-07-31T03:16:12.224326+00:00","updated_at":"2026-07-31T03:16:12.224326+00:00"},"error":null}%

# Dung server, sua tay 1 ky tu cua ciphertext_b64 truc tiep trong SQLite (byte 16: 'V' -> 'A')
→ Mini-Vault git:(main) x sqlite3 data/mini_vault.db "UPDATE kv_secrets SET ciphertext_b64 = \
replace(ciphertext_b64,'ux89ua5XBIYA+7bV','ux89ua5XBIYA+7bA') \
WHERE path='secret/alice@example.com/tamper-test';"

# Xac nhan ciphertext_b64 tren dia da bi thay doi
→ Mini-Vault git:(main) x sqlite3 data/mini_vault.db \
"select ciphertext_b64 from kv_secrets where path='secret/alice@example.com/tamper-test';"
ux89ua5XBIYA+7bATG7Rg/3w0FwjJUPuCyCxBr0BK8T5yPlwSI=%

# Restart server (DEK trong RAM mat), unlock lai, roi doc lai secret bi sua -> tu choi 100%
→ Mini-Vault git:(main) x curl -s http://localhost:8321/api/v1/kv/secret/alice@example.com/tamper-test \
-H "Authorization: Bearer gIzyIbYjY-VfZng2FwVbxSw1CKny0GMEMMTwQjaG2Xs"
{"success":false,"data":null,"error":{"code":"DECRYPTION_FAILED","message":"Secret could not be decrypted"}}%
```

Ảnh 3.3.2: Hệ thống bắt lỗi InvalidTag và từ chối giải mã khi dữ liệu trên đĩa bị can thiệp.

3.4. Phân quyền truy cập dựa trên Sở hữu (Feature 1.2)

Cấu trúc mã nguồn liên quan:

- **Authentication & Dependencies:** app/dependencies.py (Chặn token hết hạn/không hợp lệ với mã lỗi UNAUTHENTICATED)
- **Path Validation:** app/utils/validation.py (Hàm validate_path() định dạng tiền tố secret/<email>/...)
- **Business Logic:** app/kv/service.py (Cơ chế kiểm duyệt quyền sở hữu trong các hàm nghiệp vụ KVService)
- **Audit Logic:** app/router.py (Hàm denied_audit() ghi nhật ký mọi nỗ lực truy cập bất hợp pháp)

Mục tiêu cốt lõi: Thiết lập rào cản phân quyền sở hữu nhằm ngăn chặn hiện tượng đọc chéo dữ liệu giữa các định danh khác nhau, đồng thời chống lại kỹ thuật dò quét đường dẫn Path Enumeration.

Kiến trúc kiểm duyệt phân quyền được vận hành chặt chẽ qua 3 lớp phòng ngự như sau:

a. Kiểm duyệt Session Token (UNAUTHENTICATED): Trước khi bất kỳ yêu cầu nào chạm đến tầng logic, lớp Middleware trong app/dependencies.py sẽ đánh giá tính hợp lệ của Session Token. Bất kỳ token nào định dạng sai hoặc không tồn tại trong cơ sở dữ liệu đều bị từ chối ngay lập tức với mã lỗi chung chung UNAUTHENTICATED nhằm không tiết lộ thêm thông tin cho kẻ tấn công. Tuy nhiên, với thiết kế chú trọng trải nghiệm người dùng và tối ưu hóa API, riêng trường hợp Token đã quá hạn thời gian sống TTL, hệ thống sẽ chủ động ném ra ngoại lệ chuyên biệt TOKEN_EXPIRED. Điều này giúp phía Client dễ dàng nhận diện trạng thái và chủ động điều hướng người dùng quay lại màn hình đăng nhập.

b. Tiền tố sở hữu và Khớp nội định danh (PERMISSION_DENIED): Mọi bí mật được lưu trữ nội bộ dưới một đường dẫn bắt buộc mang tiền tố là email của chủ sở hữu: secret/<email>/.... Khi một yêu cầu vượt qua lớp kiểm duyệt token, hàm validate_path() sẽ được kích hoạt đầu tiên để đối chiếu email lấy từ Session Token hiện tại với email nằm trong tiền tố của đường dẫn được yêu cầu. Nếu có bất kỳ sự sai lệch nào (Ví dụ: User A cố ý truy cập secret/userB@email.com/abc), hệ thống sẽ ném ngoại lệ mang mã PERMISSION_DENIED ngay lập tức, **trước khi** tiến hành bất kỳ thao tác mã hóa hay giải mã nào tại tầng nghiệp vụ.

c. Chiến lược che giấu đường dẫn và Ghi nhật ký truy cập (Audit

Log): Lỗi PERMISSION_DENIED luôn được trả về dưới dạng một lỗi chung chung, tuyệt đối không tiết lộ liệu đường dẫn mục tiêu có thực sự tồn tại trong cơ sở dữ liệu hay không. Kỹ thuật này vô hiệu hóa hoàn toàn thủ đoạn tấn công dò quét đường dẫn (Path Enumeration). Quan trọng hơn, mọi ngoại lệ vi phạm bảo mật này sẽ được router tự động bắt lại. Ngay sau đó, hàm denied_audit() nằm tại app/router.py sẽ thực thi ghi log chi tiết mọi nỗ lực truy cập bị từ chối, bao gồm hành động (READ/WRITE/DELETE), email của kẻ tấn công, và đường dẫn mục tiêu (denied path), cung cấp dấu vết cho quá trình điều tra.

```
khanhmv2281 - zsh - 1.2 - KV chặn truy cập chéo user

# Bob dùng token của CHÍNH MINH để đọc secret của Alice
→ Mini-Vault git:(main) x curl -s http://localhost:8321/api/v1/kv/secret/alice@example.com/database \
-H "Authorization: Bearer VGpuYg2_tza2_TQKMyYfUMxuuL4dH_hlkr9msbxD03w"
{"success":false,"data":null,"error":{"code":"PERMISSION_DENIED","message":"Permission denied"}}%

# không gửi token nào cả
→ Mini-Vault git:(main) x curl -s http://localhost:8321/api/v1/kv/secret/alice@example.com/database
{"success":false,"data":null,"error":{"code":"UNAUTHENTICATED","message":"Authentication required"}}%

→ Mini-Vault git:(main) x
```

Ảnh 3.4.1: Ngăn chặn truy cập chéo (PERMISSION_DENIED) khi đọc bí mật của user khác

```
khanhmv2281 - zsh - 2.4b - audit log tu chối truy cập

# Bob dùng token của CHÍNH MINH để đọc secret của Alice
→ Mini-Vault git:(main) x curl -s http://localhost:8321/api/v1/kv/secret/alice@example.com/database \
-H "Authorization: Bearer FdrnMVNG7_bQpxwrqoCwkXtqpyRiLT5y8Xfvcotx0"
{"success":false,"data":null,"error":{"code":"PERMISSION_DENIED","message":"Permission denied"}}%

# Kiểm tra audit_logs: request bị từ chối vừa rồi đã được ghi lại
→ Mini-Vault git:(main) x sqlite3 -header -column data/mini_vault.db \
"select requester_email, action, resource_type, resource_identifier, result, created_at from audit_logs order by id desc limit 1;"
requester_email  action  resource_type  resource_identifier  result  created_at
-----
bob@example.com  READ    KV_SECRET      secret/alice@example.com/database  DENIED  2026-07-31 03:15:38.340504
```

Ảnh 3.4.2: Lỗi truy cập chéo PERMISSION_DENIED và kết quả ghi nhật ký Audit Log.

3.5. Quản trị Khóa định danh (Feature 2.1)

Cấu trúc mã nguồn liên quan:

- **Routing API:** app/router.py (Nhóm endpoint /api/v1/transit/*)
- **Business Logic:** app/transit/service.py (Thực thi nghiệp vụ khởi tạo qua hàm TransitService.create_aes)
- **Cryptography:** app/core/crypto_service.py (Mã hóa bọc khóa nội bộ qua hàm encrypt())

Mục tiêu cốt lõi: Cung cấp dịch vụ quản trị Named Key cho người dùng, đảm bảo khóa bí mật được sinh ra ngẫu nhiên và quản lý an toàn tuyệt đối dưới đĩa cứng vật lý.

Sức mạnh mật mã học của khối quản trị khóa được cấu thành từ các luồng nghiệp vụ nền tảng như sau:

a. Khởi tạo Khóa đối xứng AES-256: Hệ thống cho phép người dùng khởi tạo khóa đối xứng AES-256 phục vụ mục đích mã hóa thông qua hàm `create_aes`. Thuộc tính `key_usage` được gán cố định là `ENCRYPT_DECRYPT` nhằm giới hạn chặt chẽ ngữ cảnh sử dụng khóa.

b. Cơ chế Key Wrapping: Mọi khóa mã hóa vừa sinh ra không được trả về dưới định dạng plaintext. Hàm `create_aes` lập tức sử dụng hàm `encrypt()` kết hợp với DEK bằng thuật toán AES-256-GCM để thực hiện Key Wrapping. Điểm mấu chốt của quy trình này là việc chủ động gắn kèm chuỗi Dữ liệu xác thực bổ sung (AAD - Additional Authenticated Data) mang định dạng `key:{email}:{name}` để ràng buộc chặt chẽ bản mã với ngữ cảnh sở hữu nhằm chống lại tấn công Ciphertext Relocation (Lưu ý: Khóa ở Version 1 sẽ dùng cấu trúc AAD nguyên bản, từ Version 2 trở đi AAD sẽ tự động được nhúng thêm hậu tố `:v{N}`). Các khóa này sau đó được lưu trữ vào bảng `TransitKey`, kiến tạo một Key Hierarchy cực kỳ vững chắc."

c. Liệt kê và Thu hồi Khóa (List & Revoke): Người dùng có thể giám sát tài sản mật mã của mình thông qua API liệt kê khóa. Hệ thống chỉ trả về siêu dữ liệu Metadata gồm tên khóa và mục đích sử dụng, tuyệt đối không rò rỉ bất kỳ byte khóa thực tế nào. Đồng thời, API thu hồi khóa (Revoke) cho phép người dùng đánh dấu khóa hết hạn sử dụng. Các khóa bị thu hồi sẽ bị hệ thống từ chối trong mọi thao tác mã hóa hoặc giải mã tiếp theo nhằm giới hạn rủi ro bảo mật.

d. Xử lý Ngoại lệ và Acceptance Criteria trong Code: Mọi thao tác quản trị khóa đều được thiết kế với tư duy phòng thủ, thể hiện trực tiếp qua mã nguồn:

- **Kiểm duyệt khởi tạo:** Trong hàm `create_aes`, một truy vấn `select` được thực thi trước. Nếu tên khóa đã tồn tại trong cùng namespace (cùng `owner_email`), nó lập tức ném ngoại lệ `KEY_ALREADY_EXISTS`, đáp ứng đúng tiêu chí chặn tạo khóa trùng lặp.
- **Kiểm duyệt khởi tạo - Quyết định thiết kế:** Đề bài cho phép hai lựa chọn khi tạo trùng tên khóa: ghi đè hoặc từ chối. Nhóm đã quyết định chọn phương án từ chối. Trong hàm `create_aes`, nếu tên khóa đã tồn tại trong cùng namespace (`owner_email`), hệ thống lập tức ném ngoại lệ `KEY_ALREADY_EXISTS` (mã HTTP 409). Quyết định này bảo vệ người dùng khỏi việc vô tình xóa mất khóa cũ vốn đang chứa tư liệu mã hóa quan trọng.
- **Cam kết nghiệm thu:**
 - **Truy xuất an toàn:** Hàm `list` gọi qua hàm tĩnh `metadata(key)`, hàm này chỉ trích xuất siêu dữ liệu (`key_name`, `key_usage`, `algorithm`) và bỏ qua hoàn

toàn cột `encrypted_key_material_b64`. Điều này đảm bảo hệ thống tuyệt đối không bao giờ trả về nguyên bản chuỗi byte thực tế của khóa.

- **Khóa đã thu hồi (Quyết định thiết kế Soft-delete):** Khi gọi API `revoke`, thay vì xóa vĩnh viễn, nhóm chủ động sử dụng cơ chế xóa mềm (đánh dấu thuộc tính `key.revoked_at` bằng thời gian hiện tại). Khác biệt có chủ đích này nhằm duy trì vết cho quá trình kiểm toán. Đồng thời, hàm trung chuyển nội bộ `_find` kiểm tra nếu `key.revoked_at` có dữ liệu, nó sẽ ném lỗi `KEY_NOT_FOUND`, đảm bảo các khóa đã thu hồi sẽ ngay lập tức bị hệ thống từ chối cung cấp cho bất kỳ thao tác mã hóa/giải mã nào.

```
kanhnhv2281 - zsh - 2.1 - Transit tạo & quản lý named key

→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/keys \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4fT0QA4bh30THU" \
-H "Content-Type: application/json" \
-d '{"key_name":"payment-key","key_usage":"ENCRYPT_DECRYPT"}'
{"success":true,"data":{"key_name":"payment-key","key_usage":"ENCRYPT_DECRYPT","algorithm":"AES-256-GCM","created_at":"2026-07-24T15:26:07.084153+00:00","revoked":false},"error":null}%

# tạo lại trung ten key
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/keys \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4fT0QA4bh30THU" \
-H "Content-Type: application/json" \
-d '{"key_name":"payment-key","key_usage":"ENCRYPT_DECRYPT"}'
{"success":false,"data":null,"error":{"code":"KEY_ALREADY_EXISTS","message":"Key already exists"}}%

# list_keys không được lo key that
→ Mini-Vault git:(main) x curl -s http://localhost:8321/api/v1/transit/keys -H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4fT0QA4bh30THU"
{"success":true,"data":[{"key_name":"payment-key","key_usage":"ENCRYPT_DECRYPT","algorithm":"AES-256-GCM","created_at":"2026-07-24T15:26:07.084153","revoked":false}],"error":null}%

→ Mini-Vault git:(main) x
```

Ảnh 3.5.1: Liệt kê siêu dữ liệu khóa thành công mà không làm lộ bản nguyên của khóa

```
khanhvn2281 - zsh - 3.4 - revoke named key

# Tao mot AES key rieng de test revoke
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/keys \
  -H "Authorization: Bearer gIzyIbYjY-VfZng2FwVbxSw1CKny0GMEMMTwQjaG2Xs" \
  -H "Content-Type: application/json" \
  -d '{"key_name":"temp-revoke-key","key_usage":"ENCRYPT_DECRYPT"}'
{"success":true,"data":{"key_name":"temp-revoke-key","key_usage":"ENCRYPT_DECRYPT","algorithm":"AES-256-GCM","created_at":"2026-07-31T03:15:53.570900+00:00","revoked":false},"error":null}%

# Revoke key nay
→ Mini-Vault git:(main) x curl -s -X DELETE http://localhost:8321/api/v1/transit/keys/temp-revoke-key \
  -H "Authorization: Bearer gIzyIbYjY-VfZng2FwVbxSw1CKny0GMEMMTwQjaG2Xs"
{"success":true,"data":{"revoked":true},"error":null}%

# Kiem tra cot revoked_at trong DB da co timestamp
→ Mini-Vault git:(main) x sqlite3 -header -column data/mini_vault.db \
  "select key_name, owner_email, key_usage, revoked_at from transit_keys where key_name='temp-revoke-key';"
key_name      owner_email    key_usage      revoked_at
-----
temp-revoke-key  alice@example.com ENCRYPT_DECRYPT  2026-07-31 03:15:53.581145%

# Dung key da revoke de encrypt -> bi tu Choi
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/encrypt \
  -H "Authorization: Bearer gIzyIbYjY-VfZng2FwVbxSw1CKny0GMEMMTwQjaG2Xs" \
  -H "Content-Type: application/json" \
  -d '{"key_name":"temp-revoke-key","plaintext_b64":"aGVsbG8="}'
{"success":false,"data":null,"error":{"code":"KEY_NOT_FOUND","message":"Key not found"}}%
```

Ảnh 3.5.2: Thử nghiệm vô hiệu hóa khóa và lỗi KEY_NOT_FOUND.

3.6. Dịch vụ Mã hóa và Giải mã - Transit Engine (Feature 2.2)

Cấu trúc mã nguồn liên quan:

- **Business Logic:** app/transit/service.py (Thực thi nghiệp vụ qua hàm TransitService.encrypt và TransitService.decrypt)
- **Cryptography:** app/core/crypto_service.py (Thuật toán AES-256-GCM)

Mục tiêu cốt lõi: Cung cấp mô hình Cryptography as a Service cho phép người dùng mã hóa dữ liệu mà không bao giờ phải trực tiếp nắm giữ và quản lý Secret Key.

Nội dung cốt lõi của tính năng này được hiện thực hóa thông qua các kỹ thuật mật mã chuyên sâu như sau:

a. Mã hóa văn bản rõ Khi hàm TransitService.encrypt tiếp nhận khối văn bản rõ, nó sử dụng DEK để giải mã tạm thời khóa AES của người dùng lên vùng nhớ RAM thông qua hàm nội bộ _material(). Khóa AES này sau đó được dùng để mã hóa văn bản rõ bằng

thuật toán AES-256-GCM, sinh ra một Self-describing Ciphertext mang định dạng `vault:vX:<key_name>:<ciphertext_b64>` (với vX là thể hệ hiện tại của khóa AES, ví dụ v1, v2...)."

b. Giải mã và Xác thực Khi gọi hàm `TransitService.decrypt`, hệ thống tự động bóc tách tên khóa trực tiếp từ bản mã tự mô tả. Khóa AES tương ứng được trích xuất và giải mã lên RAM. Quá trình giải mã văn bản gốc sau đó sẽ kiểm duyệt gắt gao Authentication Tag. Mọi cấu trúc bản mã bị can thiệp đều bị thuật toán AES-GCM ném ngoại lệ `InvalidTag` và từ chối giải mã.

c. Xử lý Ngoại lệ và Acceptance Criteria trong Code

- **Kiểm duyệt tồn tại và hiệu lực của khóa:** Trước khi thực thi bất kỳ thao tác encrypt hay decrypt nào, hệ thống bắt buộc gọi hàm nội bộ `_find()` để xác thực trạng thái khóa. Nếu `key_name` hoàn toàn không tồn tại, nó ném lỗi `PERMISSION_DENIED` nhằm chống dò quét. Nếu khóa tồn tại nhưng đã bị đánh dấu thu hồi (`key.revoked_at` có giá trị), nó lập tức ném lỗi `KEY_NOT_FOUND`. Cơ chế này đảm bảo tuyệt đối các khóa đã hết vòng đời hoặc không hợp pháp sẽ bị từ chối phục vụ ngay lập tức.
- **Kiểm duyệt mục đích sử dụng:** Khi hàm encrypt hoặc decrypt gọi hàm nội bộ `_find`, nó bắt buộc phải truyền vào tham số `usage="ENCRYPT_DECRYPT"`. Hàm `_find` sẽ đối chiếu `key.key_usage != usage` và ném lỗi `INVALID_KEY_USAGE` nếu hacker cố tình truyền vào tên của một khóa chuyên dùng để Ký số (`SIGN_VERIFY`).
- **Xử lý sai lệch dữ liệu:** Trong hàm decrypt, lệnh giải mã AES-GCM được bọc trong khối `try...except InvalidTag`. Bất kỳ sự thay đổi dù là nhỏ nhất trên bản mã hoặc Authentication Tag đều khiến khối lệnh bắt lấy ngoại lệ này, chặn đứng việc lộ lọt dữ liệu và ném ra lỗi `DECRYPTION_FAILED`.
- **Cam kết nghiệm thu:**
 - **Định dạng bản mã:** Hàm encrypt kết thúc bằng việc trả về chuỗi được gán tiền tố phiên bản động `"vault:v{key_version}:{name}:{b64e(nonce + ciphertext)}"`. Đề bài gợi ý định dạng `vault:<key_name>:<b64>`, tuy nhiên nhóm quyết định chủ động bổ sung thêm tiền tố phiên bản động (ví dụ v1, v2) vào cấu trúc chuỗi. Quyết định này là thiết yếu để tích hợp hoàn hảo với tính năng nâng cao Key Rotation ở mục sau, hỗ trợ tương thích ngược tuyệt đối. Kết quả trả về đáp ứng hoàn hảo cấu trúc tự mô tả.

- **Round-trip test:** Nhờ việc bóc tách chuỗi bằng `envelope.split(":", 3)` trong hàm `decrypt` để tự động xác định phiên bản (vX), tên khóa (name) và bản mã (blob), bản rõ sau khi đi qua hai hàm này chắc chắn khớp 100% với bản gốc, vượt qua thành công bài kiểm thử round-trip.

```
khanhnv2281 - zsh - 2.2 - encrypt & decrypt as a service

→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/encrypt \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4ft0QA4bhap30THU" \
-H "Content-Type: application/json" \
-d '{"key_name":"payment-key","plaintext_b64":"SGVsbG8gTWluaSBWYXVsdA=="}'
{"success":true,"data":{"ciphertext":"vault:v1:payment-key:gxw2n7kNrQWKKI8RpZQKSfoMLX3WxHedaBDyTdN7y0F6X91b1vKNL4PPQ0s="},"error":null}%

# decrypt lại, phải ra dung plaintext ban đầu
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/decrypt \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4ft0QA4bhap30THU" \
-H "Content-Type: application/json" \
-d '{"ciphertext":"vault:v1:payment-key:gxw2n7kNrQWKKI8RpZQKSfoMLX3WxHedaBDyTdN7y0F6X91b1vKNL4PPQ0s="}'
{"success":true,"data":{"plaintext_b64":"SGVsbG8gTWluaSBWYXVsdA=="},"error":null}%

# sua 1 ky tu cuoi cua ciphertext roi decrypt lai
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/decrypt \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4ft0QA4bhap30THU" \
-H "Content-Type: application/json" \
-d '{"ciphertext":"vault:v1:payment-key:gxw2n7kNrQWKKI8RpZQKSfoMLX3WxHedaBDyTdN7y0F6X91b1vKNL4PPQ0sA"}'
{"success":false,"data":null,"error":{"code":"DECRYPTION_FAILED","message":"Ciphertext could not be decrypted"}}%

# tao them 1 signing key roi CO Y dung sai loai key de encrypt
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/signing-keys \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4ft0QA4bhap30THU" \
-H "Content-Type: application/json" \
-d '{"key_name":"document-signing-key","signing_algorithm":"ED25519"}'
{"success":true,"data":{"key_name":"document-signing-key","key_usage":"SIGN_VERIFY","algorithm":"ED25519","created_at":"2026-07-24T15:26:07.143742+00:00","revoked":false},"error":null}%

→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/encrypt \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4ft0QA4bhap30THU" \
-H "Content-Type: application/json" \
-d '{"key_name":"document-signing-key","plaintext_b64":"SGVsbG8="}'
{"success":false,"data":null,"error":{"code":"INVALID_KEY_USAGE","message":"Key cannot be used for this operation"}}%

→ Mini-Vault git:(main) x
```

Ảnh 3.6.1: Mã hóa thành chuỗi tự mô tả và giải mã round-trip hoàn hảo

```
khanhvv2281 - zsh - 3.3 - invalid key usage

# Dung key SIGN_VERIFY (document-signing-key) de goi API encrypt (danh cho ENCRYPT_DECRYPT)
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/encrypt \
-H "Authorization: Bearer gIzyIbYjY-VfZng2FwVbxSw1CKny0GMEMMTwQjaG2Xs" \
-H "Content-Type: application/json" \
-d '{"key_name":"document-signing-key","plaintext_b64":"aGVsbG8="}'
{"success":false,"data":null,"error":{"code":"INVALID_KEY_USAGE","message":"Key cannot be used for t
his operation"}} %
```

Ảnh 3.6.2: Kiểm duyệt mục đích sử dụng khóa, lỗi INVALID_KEY_USAGE.

3.7. Phân quyền Truy cập Khóa định danh (Feature 2.3)

Cấu trúc mã nguồn liên quan:

- **Business Logic:** app/transit/service.py (Hàm trung chuyển nội bộ TransitService._find)
- **Audit Logic:** app/router.py (Cơ chế giám sát denied_audit)

Mục tiêu cốt lõi: Ngăn chặn tuyệt đối việc người dùng sử dụng trái phép khóa định danh của người khác, ngay cả khi họ biết chính xác tên khóa.

Kiến trúc định danh của hệ thống được vận hành dựa trên các tiêu chuẩn bảo mật khắt khe như sau:

a. Kiểm duyệt định danh và Chống dò quét: Trái tim của phân quyền Transit nằm ở hàm TransitService._find(). Lệnh truy vấn database bắt buộc phải khớp cột owner_email == email (lấy từ Session Token). Nếu không tìm thấy (do khóa không tồn tại, hoặc do khóa thuộc về user khác), đoạn code chủ đích ném chung một ngoại lệ duy nhất là PERMISSION_DENIED thay vì NOT_FOUND. Điều này khiến kẻ tấn công không thể dựa vào mã lỗi để dò quét (Path Enumeration) xem khóa đó có thực sự tồn tại trong DB hay không.

b. Ghi nhật ký Audit: Đề bài yêu cầu 100% các yêu cầu bị từ chối đều phải được ghi log. Điều này được thực thi tại app/router.py (hàm transit_call). Hàm này dùng khối try...except AppError để bắt lấy lỗi PERMISSION_DENIED văng ra từ _find, sau đó gọi ngay hàm denied_audit để ghi vào DB toàn bộ hành động (ENCRYPT/DECRYPT/SIGN...), email kẻ tấn công, và khóa mục tiêu bị từ chối.

```
khanhmv2281 - zsh - 2.3 - Transit chan truy cap cheo user

# Bob dung dung key_name cua Alice de encrypt
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/encrypt \
-H "Authorization: Bearer VGpuYg2_tza2_TQKMyYfUMxuUL4dH_hlkR9msbxD03w" \
-H "Content-Type: application/json" \
-d '{"key_name":"payment-key","plaintext_b64":"SGVsbG8="}'
{"success":false,"data":null,"error":{"code":"PERMISSION_DENIED","message":"Permission denied"}}%
→ Mini-Vault git:(main) x
```

Ảnh 3.7.1: Từ chối thao tác (PERMISSION_DENIED) khi sử dụng khóa của người khác

3.8. Dịch vụ Ký và Xác thực Chữ ký số (Feature 2.4)

Cấu trúc mã nguồn liên quan:

- **Business Logic:** app/transit/service.py (Thực thi nghiệp vụ qua các hàm create_signing, sign, verify)
- **Cryptography:** Thư viện cryptography.hazmat (Xử lý đường cong elliptic Ed25519)

Mục tiêu cốt lõi: Cung cấp dịch vụ ký điện tử an toàn, đảm bảo tính chống chối bỏ và toàn vẹn của thông điệp thông qua hạ tầng PKI.

Sức mạnh mật mã học của cỗ máy chữ ký số được cấu thành từ các luồng nghiệp vụ nền tảng như sau:

a. Khởi tạo Cặp khóa Bất đối xứng: Thông qua hàm create_signing, hệ thống phát sinh cặp khóa theo chuẩn đường cong elliptic Ed25519 mang lại hiệu năng cao. Private Key lập tức bị Key Wrapping bằng DEK trước khi lưu vào cơ sở dữ liệu. Public Key được lưu ở định dạng chuẩn phục vụ xác thực nội bộ.

b. Quy trình Ký số và Xác thực Chữ ký: Khi hàm TransitService.sign được gọi, khóa riêng được giải mã tạm thời lên RAM để ký trực tiếp lên thông điệp băm, thuộc tính key_usage được kiểm duyệt nghiêm ngặt phải khớp với SIGN_VERIFY. Ngược lại, hàm TransitService.verify thực thi xác thực bằng Khóa công khai. Bất kỳ sự xáo trộn dù chỉ một bit trên thông điệp gốc cũng sẽ làm hỏng cấu trúc toán học của đường cong elliptic, dẫn đến ngoại lệ InvalidSignature và kết quả trả về không hợp lệ.

c. Quyết định thiết kế: Tiền xử lý Thông điệp (RAW vs DIGEST): Hệ thống xử lý thông điệp theo định dạng (message_type) dựa trên đặc tả kỹ thuật cốt lõi:

- **Trường hợp RAW:** Nhóm quyết định ký thẳng thông điệp gốc và không băm trước bằng SHA-256. Lý do là vì kiến trúc Ed25519 (PureEdDSA) đã tự động băm nội bộ bằng SHA-512. Đồng thời, thư viện cryptography cũng không cung cấp chế độ prehash cho Ed25519.
- **Trường hợp DIGEST:** Khi client tự băm thông điệp bằng SHA-256, hệ thống sẽ kiểm duyệt gắt gao độ dài bắt buộc phải là 32-bytes (ném `INVALID_DIGEST_LENGTH` nếu sai chuẩn). Sau đó, hệ thống coi chính khối mã băm này như một thông điệp thông thường để đưa vào hàm ký Ed25519.

d. Quyết định thiết kế: Thuật toán Ký (signing_algorithm): Nhóm quyết định tối ưu hóa kiến trúc bằng việc **chỉ hỗ trợ duy nhất thuật toán ED25519**. Hàm verify tiếp nhận thêm tham số tùy chọn `signing_algorithm`. Nếu client truyền vào tham số này nhưng giá trị không khớp với thuật toán thực tế của khóa, hệ thống lập tức ném ngoại lệ `INVALID_SIGNING_ALGORITHM`, hoàn toàn tuân thủ bắt lỗi error case của đề bài.

e. Xử lý Ngoại lệ và Acceptance Criteria trong Code

- **Kiểm duyệt độ dài mã băm:** Cả hai hàm `sign` và `verify` đều có một khối logic đặc thù: `if message_type == "DIGEST" and len(message) != 32`. Logic này đảm bảo hệ thống chặn đứng các mã băm sai chuẩn trước khi đưa vào đường cong elliptic, ném ra chính xác mã lỗi `INVALID_DIGEST_LENGTH`.
- **Xử lý chữ ký sai lệch:** Trong hàm `verify`, quá trình dùng Khóa công khai (`Ed25519PublicKey`) để xác thực chữ ký được đặt gọn trong khối try...except `InvalidSignature`. Nếu bất kỳ bit nào của chữ ký hoặc thông điệp bị thay đổi trên đường truyền, kết quả bắt lỗi sẽ gán cờ `valid = False` và trả về một cách an toàn mà không làm sập server.
- **Cam kết nghiệm thu:**
 - **Xác thực đa định dạng:** Hệ thống phân nhánh thông minh dựa vào `message_type` ở đầu vào API, giúp chữ ký số sinh ra đồng nhất tính đúng đắn bất kể thông điệp ở dạng thô RAW hay đã băm DIGEST.
 - **Tính chống chối bỏ:** Private Key sinh ra từ `Ed25519PrivateKey.generate()` lập tức bị khóa kín bởi thao tác Key Wrapping (mã hóa bọc bằng DEK) trước khi xuống DB. Kẻ tấn công không thể có được Private Key (ẩn danh hoàn toàn) để giả mạo chữ ký, trong khi Public Key lại có thể dễ dàng truy xuất để phía thứ ba xác minh.

```
khanhnhv2281 - zsh - 2.4 - sign & verify

→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/sign \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4fT0QA4bhap30THU" \
-H "Content-Type: application/json" \
-d '{"key_name":"document-signing-key","message_b64":"SGVsbG8gTWluaSBWYXVsdA==","message_type":"RAW"}'
{"success":true,"data":{"key_name":"document-signing-key","signature_b64":"CThTXz1D1yfPEV8iGsSVIMBG4lKWqYiwQCsQpmf/xioD9FK7qzyJNrXZljX76Kkcoizqm9SDK8mMJ8Bqm4IbAw==","signing_algorithm":"ED25519"},"error":null}%

# verify voi message goc
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/verify \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4fT0QA4bhap30THU" \
-H "Content-Type: application/json" \
-d '{"key_name":"document-signing-key","message_b64":"SGVsbG8gTWluaSBWYXVsdA==","message_type":"RAW","signature_b64":"CThTXz1D1yfPEV8iGsSVIMBG4lKWqYiwQCsQpmf/xioD9FK7qzyJNrXZljX76Kkcoizqm9SDK8mMJ8Bqm4IbAw=="}'
{"success":true,"data":{"key_name":"document-signing-key","signature_valid":true,"signing_algorithm":"ED25519"},"error":null}%

# sua 1 ky tu cuoi cua message_b64 roi verify lai (van dung chu ky cu)
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/verify \
-H "Authorization: Bearer 3ZfkMX_bzP1kgiF3RbZ3VYaK4S4fT0QA4bhap30THU" \
-H "Content-Type: application/json" \
-d '{"key_name":"document-signing-key","message_b64":"SGVsbG8gTWluaSBWYXVsdD8=","message_type":"RAW","signature_b64":"CThTXz1D1yfPEV8iGsSVIMBG4lKWqYiwQCsQpmf/xioD9FK7qzyJNrXZljX76Kkcoizqm9SDK8mMJ8Bqm4IbAw=="}'
{"success":true,"data":{"key_name":"document-signing-key","signature_valid":false,"signing_algorithm":"ED25519"},"error":null}%

→ Mini-Vault git:(main) x
```

Ảnh 3.8.1: Sử dụng khóa Ed25519 để ký thành công và xác thực tính toàn vẹn

```
khanhnhv2281 - zsh - 3.5b - invalid digest length

# Goi sign voi message_type=DIGEST nhưng message chỉ có 5 byte (không phải 32 byte SHA-256)
→ Mini-Vault git:(main) x curl -s -X POST http://localhost:8321/api/v1/transit/sign \
-H "Authorization: Bearer gIzyIbYjY-VfZng2FwVbxSw1CKny0GMEMMTwQjaG2Xs" \
-H "Content-Type: application/json" \
-d '{"key_name":"document-signing-key","message_b64":"aGVsbG8=","message_type":"DIGEST"}'
{"success":false,"data":null,"error":{"code":"INVALID_DIGEST_LENGTH","message":"Digest must be 32 bytes"}}%
```

Ảnh 3.8.2: Bắt lỗi INVALID_DIGEST_LENGTH khi payload không đủ 32-bytes.

4. Tính năng Extra Credit

Bên cạnh các yêu cầu cơ bản của đề bài, nhóm đã chủ động nghiên cứu và triển khai thành công 3 tính năng nâng cao nhằm hoàn thiện hệ sinh thái bảo mật của Mini-Vault, đưa kiến trúc hệ thống tiệm cận với các tiêu chuẩn công nghiệp hiện đại. Các tính năng này bao gồm: **Secret Versioning** (Lưu trữ và phục hồi lịch sử thay đổi bí mật), **Key Rotation** (Luân chuyển khóa mã hóa động để giảm thiểu rủi ro lộ lọt) và **Tamper-evident Audit Log** (Nhật ký kiểm soát kháng sửa đổi sử dụng cơ chế Hash-chaining).

4.1. Secret Versioning

Cấu trúc mã nguồn liên quan:

- **Routing API:** app/router.py (Endpoint GET /api/v1/kv-versions và GET /api/v1/kv/{path}?version=X)
- **Business Logic:** app/kv/service.py (Thực thi logic versioning qua hàm KVService.versions và KVService.read)
- **Database Models:** app/models.py (Bảng KVSecretVersion)

Chi tiết Kỹ thuật và Quyết định Thiết kế:

- **Mô hình lưu trữ Head-and-History:** Nhóm quyết định không dùng chung một bảng duy nhất để tránh chi phí truy vấn phức tạp. Bảng KVSecret đóng vai trò là HEAD pointer luôn lưu bản ghi mới nhất giúp tối ưu tốc độ đọc dữ liệu hiện tại ở mức $O(1)$. Bảng phụ KVSecretVersion hoạt động như một Append-Only Ledger, tự động lưu lại toàn bộ lịch sử mỗi khi có thao tác Write.
- **Cơ chế gia tăng phiên bản:** Khi gọi API Write lên một đường dẫn đã tồn tại, hệ thống thực thi lệnh select để lấy version cao nhất hiện tại, cộng thêm 1, thay vì sử dụng một lệnh UPDATE ghi đè làm mất vĩnh viễn dữ liệu cũ theo cách truyền thống, hệ thống thực thi một Transaction bao gồm 2 thao tác đồng thời: (1) Cập nhật bảng HEAD pointer KVSecret với bản mã mới nhất nhằm duy trì tốc độ truy xuất $O(1)$, và (2) Chèn thêm một bản ghi mới mang bản mã này vào bảng KVSecretVersion. Kiến trúc này vừa đảm bảo hiệu năng đọc dữ liệu hiện tại, vừa thiết lập được một Append-Only Ledger bất biến cho lịch sử phiên bản.

- **Cơ chế Time Travel:** Tham số tùy chọn ?version=X được thêm vào API Read. Khi tham số này xuất hiện, lệnh truy vấn sẽ chuyển hướng thẳng vào bảng KVSecretVersion. Nếu X không tồn tại, hệ thống ném ngoại lệ NOT_FOUND (HTTP 404) thay vì trả về version gần nhất, đảm bảo tính chính xác tuyệt đối của yêu cầu truy xuất.

```
move-Item data\*.db -Force -ErrorAction SilentlyContinue; Write-Host "`n--- DANG CHAY DEMO FULL ---`n" -ForegroundColor Cyan; powershell -ExecutionPolicy Bypass -File scripts\demo_full.ps1; Write-Host "`n--- DANG CHAY DEMO NANG CAO ---`n" -ForegroundColor Cyan; powershell -ExecutionPolicy Bypass -File scripts\demo_advanced.ps1moveItemCommand
>> env) PS D:\An ninh máy tính\project 1\Mini-Vault>

[*] Khoi dong server ao tren port 8200...
[*] Cho server khoi dong...

=== ADVANCED 1: SECRET VERSIONING (Luu nhieu phien ban cua 1 Secret) ===
> PUT /api/v1/kv/secret/alice@example.com/api-key
{
  "success": true,
  "data": {
    "path": "secret/alice@example.com/api-key",
    "version": 1,
    "created_at": "2026-07-31T09:05:46.073639+00:00",
    "updated_at": "2026-07-31T09:05:46.073639+00:00"
  },
  "error": null
}
> PUT /api/v1/kv/secret/alice@example.com/api-key
{
  "success": true,
  "data": {
    "path": "secret/alice@example.com/api-key",
    "version": 2,
    "created_at": "2026-07-31T09:05:46.073639",
    "updated_at": "2026-07-31T09:05:46.105298+00:00"
  },
  "error": null
}
```

Ảnh 4.1.1: Quá trình tạo mới và ghi nhận liên tiếp 2 phiên bản (Version 1 và 2) cho cùng một Secret

```

> Doc Version 1 (cu):
> GET /api/v1/kv/secret/alice@example.com/api-key?version=1
{
  "success": true,
  "data": {
    "path": "secret/alice@example.com/api-key",
    "version": 1,
    "value": {
      "key": "AAAAA"
    }
  },
  "error": null
}

> Doc Version 2 (moi):
> GET /api/v1/kv/secret/alice@example.com/api-key?version=2
{
  "success": true,
  "data": {
    "path": "secret/alice@example.com/api-key",
    "version": 2,
    "value": {
      "key": "BBBBB"
    }
  },
  "error": null
}

```

Ảnh 4.1.2: Kết quả truy xuất độc lập từng phiên bản Secret dựa trên tham số version

Hệ thống ghi nhận và duy trì thành công 2 phiên bản cho cùng một Secret, cho phép truy xuất độc lập từng phiên bản thông qua tham số ?version=X.

4.2. Key Rotation

Cấu trúc mã nguồn liên quan:

- **Routing API:** app/router.py (Endpoint POST /api/v1/transit/keys/{name}/rotate)
- **Business Logic:** app/transit/service.py (Hàm TransitService.rotate sinh khóa mới và TransitService._material tự động dò version)
- **Database Models:** app/models.py (Bảng TransitKeyVersion)

Chi tiết Kỹ thuật và Quyết định Thiết kế:

- **Cơ chế kích hoạt luân chuyển:** Luân chuyển khóa không chạy ngầm qua cronjob mà được thiết kế dưới dạng Manual Explicit Rotation API. Quản trị viên sẽ gọi API này thủ công tùy theo chính sách bảo mật nội bộ (ví dụ: định kỳ 90 ngày) hoặc gọi ngay lập tức khi nghi ngờ có sự cố lộ lọt khóa cũ.

- **Thuật toán phát sinh khóa mới:** Hàm rotate không tái sử dụng bất kỳ entropy nào từ khóa cũ. Nó gọi xuống trình sinh số ngẫu nhiên của hệ điều hành `os.urandom(32)` để phát sinh 256-bit Key Material hoàn toàn mới.
- **Cơ chế Cryptographic Binding:** Đây là thiết kế bảo mật mấu chốt. Số định danh version (ví dụ: v2) được nhúng cứng vào chuỗi Additional Authenticated Data trong 2 giai đoạn:
 1. Khi bọc khóa: Dùng chuỗi `key:{email}:{name}:v2`.
 2. Khi mã hóa dữ liệu: Dùng chuỗi `transit:{email}:{name}:v2`. Thiết kế này khóa chặt Ciphertext vật lý với đúng thể hệ version của nó, vô hiệu hóa hoàn toàn kỹ thuật Swap Attack giữa các phiên bản.
- **Cơ chế Backward Compatibility:** Định dạng Transit Ciphertext sinh ra luôn được gắn tiền tố mô tả version (`vault:v2...`). Trong hàm decrypt, hệ thống tự động bóc tách chuỗi bằng lệnh `split(":")`, đọc ra tham số vX, rồi query đúng Key Material cũ ở bảng `TransitKeyVersion` để tiến hành giải mã. Nhờ đó, dù khóa đã rotate 100 lần, dữ liệu cũ mã hóa từ version 1 vẫn giải mã thành công ngay lập tức mà không cần bất kỳ tiến trình Data Migration nào.

```

=== ADVANCED 2: KEY ROTATION (Xoay vòng khóa Transit) ===
> POST /api/v1/transit/keys
{
  "success": true,
  "data": {
    "key_name": "master-key",
    "key_usage": "ENCRYPT_DECRYPT",
    "algorithm": "AES-256-GCM",
    "current_version": 1,
    "created_at": "2026-07-31T09:05:46.171015+00:00",
    "revoked": false
  },
  "error": null
}
> Mã hóa lần 1 (Version 1): vault:v1:master-key:1SPVd1ZMYtDp4PJVhFTClwuoLYb9TTHJxH9KT3MF657p9vUN

> Xoay vòng khóa (Rotate) lên Version 2:
> POST /api/v1/transit/keys/master-key/rotate
{
  "success": true,
  "data": {
    "key_name": "master-key",
    "key_usage": "ENCRYPT_DECRYPT",
    "algorithm": "AES-256-GCM",
    "current_version": 2,
    "created_at": "2026-07-31T09:05:46.171015",
    "revoked": false
  },
  "error": null
}
> Mã hóa lần 2 (Version 2): vault:v2:master-key:aBlZFm6IrKajSRXe0MHihN3ne5V00My1axICYGq5z2xgp0y

```

Ảnh 4.2.1: Quá trình sinh khóa Transit V1 để mã hóa dữ liệu và tiến hành xoay vòng khóa lên V2

```
> Giải mã lại ciphertext V1 (Hệ thống tự động nhận diện dung khóa V1 để giải mã):  
> POST /api/v1/transit/decrypt  
{  
  "success": true,  
  "data": {  
    "plaintext_b64": "SGVsbG8gVjE="
```

Ảnh 4.2.2: Hệ thống tự động nhận diện và giải mã thành công bản mã cũ (V1) dù khóa mặc định đã là V2

4.3. Tamper-evident Audit Log

Cấu trúc mã nguồn liên quan:

- **Routing API:** app/router.py (Endpoint GET /api/v1/audit/verify)
- **Business Logic:** app/audit/service.py (Hàm audit quản lý ghi log và cơ chế khóa Lock)
- **Database Models:** app/models.py (Bảng AuditLog mở rộng với cột hash_b64 và previous_hash_b64)

Chi tiết Kỹ thuật và Quyết định Thiết kế:

- **Cơ chế Hash-chaining:** Lấy cảm hứng từ cấu trúc dữ liệu của Blockchain, bảng audit_logs được thiết kế sao cho mỗi dòng log sinh ra sẽ lưu trữ 2 mã băm mật mã: previous_hash_b64 (mã băm của dòng log liền kề trước đó) và hash_b64 (mã băm của chính dòng hiện tại). Bản ghi đầu tiên trong chuỗi mang giá trị mặc định là "GENESIS".
- **Hiệu ứng thác nước trong kiểm duyệt:** Khi gọi API /audit/verify, hệ thống sẽ quét tuần tự toàn bộ bảng log từ đầu. Quá trình kiểm duyệt tự động tính toán lại mã SHA-256 của từng dòng dựa trên các trường lỗi (như id, email, action, result) và nối với previous_hash. Nếu kẻ tấn công dùng lệnh SQL can thiệp trực tiếp vào file CSDL như đổi giá trị DENIED thành GRANTED, mã băm của dòng đó sẽ thay đổi hoàn toàn, lập tức phá vỡ tính khớp nối của mắt xích ở dòng kế tiếp. API lập tức dừng lại, báo động lỗi Hash mismatch kèm chính xác ID dòng dữ liệu bị can thiệp trái phép.

- **Công thức tính mã băm (SHA-256):** Hệ thống tiến hành nối các trường dữ liệu nghiệp vụ bằng ký tự phân cách | để tạo thành chuỗi đầu vào:

```
data_str = f'{id}|{prev_hash}|{email}|{action}|{identifier}|{result}'
```

Chuỗi dữ liệu này sau đó được đưa qua hàm băm mật mã học SHA-256 và lưu xuống cơ sở dữ liệu dưới định dạng Base64. Cơ chế này đảm bảo mọi thay đổi dù là nhỏ nhất cũng phản ánh lên kết quả băm cuối cùng.

- **Kiểm soát tương tranh:** Để ngăn chặn triệt để lỗi Race Condition - xảy ra khi nhiều request cố gắng ghi log cùng một phần nghìn giây, dẫn đến việc đọc chung một previous_hash và làm phân nhánh chuỗi băm - quá trình ghi nhật ký được đưa vào vùng tranh chấp tới hạn bằng threading.Lock(). Điều này ép buộc hệ điều hành xử lý các luồng thực thi một cách tuần tự, hy sinh một phần nhỏ hiệu năng để đổi lấy tính toàn vẹn tuyệt đối của chuỗi log.

```
=== ADVANCED 3: TAMPER-EVIDENT AUDIT LOG (Phát hiện sửa đổi Log) ===
> Xác minh chuỗi log (Trước khi bị xâm phạm):
> GET /api/v1/audit/verify
{
  "success": true,
  "data": {
    "valid": true
  },
  "error": null
}
```

Ảnh 4.3.1: Xác minh tính toàn vẹn của chuỗi log hợp lệ trước khi bị Hacker can thiệp

```
> [HACKER ATTACK] Dùng Python để lên sửa CSDL (Đổi DENIED thành GRANTED ở dòng log cuối cùng)...  
[OK] Hacker đã sửa CSDL thành công.  
  
> Xác minh lại chuỗi log (Sau khi bị xâm phạm):  
> GET /api/v1/audit/verify  
{  
  "success": true,  
  "data": {  
    "valid": false,  
    "broken_at_id": 1,  
    "reason": "Hash mismatch"  
  },  
  "error": null  
}  
  
> Xác minh lại chuỗi log (Sau khi bị xâm phạm):  
> GET /api/v1/audit/verify  
○ > Xác minh lại chuỗi log (Sau khi bị xâm phạm):  
> GET /api/v1/audit/verify  
{  
  "success": true,  
  "data": {  
    "valid": false,  
    "broken_at_id": 1,  
    "reason": "Hash mismatch"  
  },  
  "error": null  
}
```

Ảnh 4.3.2: Hệ thống phát hiện tức thời dữ liệu bị sửa đổi trái phép dưới cơ sở dữ liệu nhờ đứt gãy chuỗi băm

5. Kết luận và Giới hạn triển khai

5.1. Tổng kết

Dự án Mini-Vault đã hiện thực hóa thành công một hệ thống quản trị định danh và mật mã học tập trung theo mô hình Cryptography-as-a-Service, đáp ứng toàn vẹn các tiêu chuẩn OWASP và NIST. Kiến trúc hệ thống tuân thủ nghiêm ngặt nguyên lý Least Privilege và Defense in Depth, thiết lập sự cô lập tuyệt đối giữa logic định tuyến, phân quyền và lỗi mật mã.

Các kỹ thuật mật mã hiện đại bao gồm Argon2id, mã hóa AEAD AES-256-GCM và chữ ký số đường cong elliptic Ed25519 được tích hợp nguyên bản với cơ chế Key Wrapping đa tầng, đảm bảo tiêu chí Encrypted-at-Rest tại Storage Layer. Hệ thống xử lý triệt để các rủi ro bảo mật thông qua cơ chế chối bỏ truy cập Generic Error, vô hiệu hóa hoàn toàn kỹ thuật Path Enumeration và Credential Stuffing, đồng thời cam kết Data Encryption Key không bao giờ bị rò rỉ dưới định dạng plaintext xuống bộ nhớ vật lý.

5.2. Giới hạn triển khai

Mặc dù đạt mức độ hoàn thiện cao về mặt logic mật mã, nguyên mẫu hiện tại vẫn tồn tại một số giới hạn đặc thù thuộc về kiến trúc nền tảng và môi trường thực thi:

- **Cơ chế Garbage Collection:** Việc giải phóng vùng nhớ chứa Data Encryption Key khỏi RAM phụ thuộc hoàn toàn vào trình quản lý bộ nhớ tự động của runtime Python. Trái ngược với các ngôn ngữ biên dịch bậc thấp, Python không hỗ trợ API zeroize memory mức phân cứng. Đặc tính này kiến tạo một độ trễ hữu hạn từ thời điểm lệnh Lock Vault được gọi cho đến khi tham chiếu vùng nhớ thực sự bị hủy bỏ, để ngỏ rủi ro Memory Dump trong các kịch bản tấn công vật lý cục bộ.
- **Mô hình lưu trữ đơn điểm:** Việc ứng dụng SQLite giới hạn hệ thống ở kiến trúc Single-Instance. Khi ứng dụng kỹ thuật Scale-out thông qua worker pool trên Uvicorn, cơ chế lưu trữ DEK tại RAM nội bộ của từng process sẽ gây ra hiện tượng phân mảnh trạng thái Vault. Môi trường Production thực tế đòi hỏi sự dịch chuyển sang các hệ quản trị cơ sở dữ liệu phân tán kết hợp với giao thức Unseal đồng thuận đa điểm.

Tài liệu tham khảo

- [1] HashiCorp, "Transit Secrets Engine", *HashiCorp Vault Documentation*. [Trực tuyến]. Trích xuất từ: <https://developer.hashicorp.com/vault/docs/secrets/transit>
- [2] Amazon Web Services, "CreateKey", *AWS Key Management Service API Reference*. [Trực tuyến]. Trích xuất từ: https://docs.aws.amazon.com/kms/latest/APIReference/API_CreateKey.html
- [3] Amazon Web Services, "Sign" và "Verify", *AWS Key Management Service API Reference*. [Trực tuyến]. Trích xuất từ: https://docs.aws.amazon.com/kms/latest/APIReference/API_Sign.html và https://docs.aws.amazon.com/kms/latest/APIReference/API_Verify.html
- [4] Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST), "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)", *NIST Special Publication 800-38D*, 2007.
- [5] Python Cryptographic Authority, "cryptography (hazmat primitives)", thư viện mã nguồn mở hỗ trợ thuật toán AES-GCM và đường cong elliptic Ed25519.